

Greentiq Advanced CRM

Comprehensive Project Technical Documentation & Candidate Assessment Reference

Candidate Profile & Assessment Submission

My name is **Akash KS**, a Frontend Developer with around 2 years of experience in frontend development. This document details the complete technical implementation of the assessment provided for the Frontend Developer position at **Greentiq**. I am looking forward to the opportunity to work with the team at Greentiq.

Contact Details:

Phone: 8861291864
Email: akashks.dev@gmail.com
GitHub: <https://github.com/thelogicnomad>

Project Name: Greentiq Advanced CRM

Framework: Next.js 15.1.7 (App Router)

Language: TypeScript 5.7.3 (Strict)

Styling: Tailwind CSS 3.4 + Custom Theme

State & Persistence: TanStack Query v5 + Upstash Redis

Build Output: Static & Server App Router

Table of Contents

- 1. Overview
- 2. Tech Stack & Dependencies
- 3. Architecture & Data Flow
- 4. Getting Started & Operations
- 5. Comprehensive Feature Breakdown
- 6. Design System & UI Specifications
- 7. Annotated Project Structure
- 8. API & Data Access Layer
- 9. Deployment & Build Configuration
- 10. Known Limitations & Architecture Notes

1. Overview

Greentiq Advanced CRM is a production-grade, highly responsive Customer Relationship Management (CRM) web application. Built with Next.js 15 and TypeScript, it is designed for sales teams and administrators to manage customer profiles, track deals, filter sales pipelines with complex multi-criteria logic, and maintain detailed interaction timelines.

Target Users: CRM Administrators, Sales Representatives, Account Managers, and Operations Leads.

2. Tech Stack & Dependencies

Package / Tool	Version	Role & Purpose
Next.js	15.1.7	React framework providing App Router, SSG, and Node Route Handlers.
React & React DOM	19.0.0	Core UI library rendering component trees and handling state.
TypeScript	5.7.3	Strict static typing across types, APIs, and UI components.
@upstash/redis	1.34.5	Serverless Redis client for persistent Vercel production storage.
TanStack Query	5.66.9	Client-side server-state management, caching, and optimistic updates.
TanStack Virtual	3.13.0	DOM virtualization engine for high-performance Card layout rendering.
Tailwind CSS	3.4.17	Utility-first styling system with custom CSS token mapping.
shadcn/ui & Radix	Latest	Accessible, unstyled UI primitives (Dialog, Select, Popover, Dropdown).
dnd-kit	6.3.1	Drag-and-drop sortable list implementation for saved filters.
React Hook Form + Zod	7.54 / 3.24	Type-safe form handling and schema validation for customer payloads.
@faker-js/faker	9.5.0	Deterministic seed generation for 150 mock customer records.
date-fns	4.1.0	Date formatting, manipulation, and range filtering utilities.
sonner	2.0.1	Toast notification manager with customized status indicators.
use-debounce	10.0.4	300ms debouncing hook for real-time search queries.

3. Architecture & Data Flow

The application follows a clean, modular architecture combining custom hooks, barrel exports, and isolated API handlers with persistent Redis storage:

- 1. Component Level:** UI components in `src/components/crm/` consume props and trigger callbacks. All custom components follow a single-component-per-folder structure with `index.ts` barrel exports.
- 2. Hook Layer:** Encapsulated custom hooks in `src/hooks/` (e.g., `useCustomers`, `useCustomerMutations`, `useSavedFilters`) wrapping TanStack Query queries and mutations.
- 3. API Client Layer:** Client functions in `src/lib/api/client.ts` make fetch requests to Next.js route handlers under `app/api/`.
- 4. Persistence & Data Access:** Next.js App Router handlers interact with `src/lib/api/db.ts`. In production, mutations persist server-side via `@upstash/redis` (keys: `crm:customers`, `crm:saved-filters`). For local dev, zero-config fallback uses `globalThis`.

4. Getting Started & Operations

Prerequisites: Node.js $\geq 18.x$, npm $\geq 9.x$.

Installation Requirement (FIRST STEP): You must run `npm install` (or `npm install --legacy-peer-deps`) first to install all required dependencies before starting the development server or executing scripts.

Environment Setup: Pull production Redis credentials using `vercel env pull .env.local`. If no env vars are pulled, local development seamlessly falls back to the in-memory store.

Command Execution Sequence:

```
# Step 1: Install Dependencies (REQUIRED FIRST)
npm install

# Step 2: Pull Vercel Environment Variables (Optional for Redis)
vercel env pull .env.local

# Step 3: Run Development Server
npm run dev

# Step 4: Build Production Bundle
npm run build

# Step 5: Start Production Server
npm start

# Step 6: Execute Automated Filter Test Suite
npm run test:filters
```

5. Comprehensive Feature Breakdown

- **Customer List Dual Layouts (Table & Virtualized Card View) (app/page.tsx & src/components/crm/CustomerTable/)**

Displays the customer list in both Table layout (for desktop grid inspection with column sorting and pagination) and Card layout. The Card layout utilizes DOM virtualization (@tanstack/react-virtual) for high-performance windowed rendering during infinite scrolling. Supports real-time debounced search (300ms via use-debounce) across Name, Email, and Company, with responsive view auto-switching (< sm breakpoint).

- **Advanced Filters Panel & Drag-and-Drop Presets (src/components/crm/AdvancedFiltersSheet/)**

Multi-criteria filtering including status checkboxes, company multi-select dropdown with removable tags, date range pickers (date-fns), and partial string matches for phone/email. Includes saved filter presets reorderable via @dnd-kit/core and @dnd-kit/sortable.

- **Interactive Customer Details & Timeline (src/components/crm/CustomerDetailsModal/)**

Drawer displaying customer metadata, deal value, account owner, and interactive notes timeline. Supports live note creation via TanStack Query mutations with immediate cache refetching.

- **Add & Edit Customer Modal with Zod Validation (src/components/crm/CustomerFormModal/)**

Form dialog built with react-hook-form and Zod schema validation (validates required name, email format, phone, and positive deal value). Returns inline field-level validation errors.

- **Optimistic Customer Deletion (src/components/crm/DeleteConfirmDialog/)**

Confirmation modal triggering optimistic deletion. Immediately removes customer from TanStack Query cache with rollback capability on mutation failure.

- **Bulk Selection & Multi-Customer Actions (src/components/crm/BulkActionsBar/)**

Floating bulk actions toolbar allowing multi-customer selection across pages, bulk status mutation, bulk deletion, and bulk CSV export.

- **CSV Data Export (app/page.tsx)**

Export function generating formatted CSV documents from currently filtered customer datasets, preserving search and status filters.

- **Global Command Palette & Keyboard Shortcuts (src/components/crm/KeyboardShortcutsHelp/)**

Global shortcut listener (Cmd+K for command palette, Cmd+F for filters panel, Cmd+Shift+A for Add Customer, ? for shortcut guide). Features focus-collision prevention when typing inside inputs.

6. Design System & UI Specifications

Modern Custom Dual Theme: The application supports both Dark and Light themes defined in app/globals.css:

- **Dark Theme:** Hex color tokens (#18191c background, #1e2022 card, #5e6ad2 primary brand purple, #e4e5e9 foreground text).
- **Light Theme:** Perceptually uniform CSS lch() color tokens (lch(94.44% 0.5 282) background, lch(48% 59.31 288.43) primary brand purple).
- **Status Badges:** CSS custom properties (--status-active-*, --status-prospect-*, --status-lead-*, --status-inactive-*, --status-archived-*) adapting badge text and background opacity automatically.
- **Hydration Safety:** ThemeToggle uses a mounted state guard to prevent SSR hydration mismatches.

7. Annotated Project Structure

```
greentiq/
app/
api/ # Next.js App Router API Route Handlers
customers/ # CRUD handlers for customer entities
saved-filters/ # CRUD and reorder handlers for saved filters
globals.css # Custom dual theme CSS custom properties
layout.tsx # Root HTML wrapper and font provider
page.tsx # Main CRM Dashboard page container
providers.tsx # Client providers (TanStack Query, ThemeProvider, Sonner)
src/
components/
crm/ # Feature-specific CRM components (Folder-per-component)
AdvancedFiltersSheet/
AppSidebar/
BulkActionsBar/
CustomerCard/
CustomerDetailsModal/
CustomerFormModal/
CustomerTable/
KeyboardShortcutsHelp/
TopNavbar/
ui/ # Accessible shadcn/ui primitive components
hooks/ # Custom React hooks (TanStack Query integrations)
lib/
api/ # Upstash Redis DB (db.ts), Seed (seed.ts), Client (client.ts)
utils.ts # Formatting, badge variant, and helper utilities
scripts/
test-filters.ts # Comprehensive automated filter composition test suite
types/ # TypeScript domain models and filter type schemas
public/
greentiq-logo.png # Greentiq brand logo asset
profile.png # User profile avatar portrait asset
```

8. API & Data Access Layer

The API layer persists data via Upstash Redis (with simulated latency of 300ms–600ms):

Endpoint	Method	Description
GET /api/customers	GET	Paginated/Infinite fetch with search, status, company, date range, phone, and sorting filters.
POST /api/customers	POST	Creates customer and persists payload to Redis (crm:customers).
GET /api/customers/[id]	GET	Fetches single customer record details.
PATCH /api/customers/[id]	PATCH	Partial update for customer details or status, persisted to Redis.
DELETE /api/customers/[id]	DELETE	Deletes customer entity from Redis store.
POST /api/customers/[id]/notes	POST	Appends interaction note to customer timeline in Redis.
GET /api/saved-filters	GET	Lists all saved filter presets ordered by index.
POST /api/saved-filters	POST	Saves new filter preset to Redis (crm:saved-filters).
POST /api/saved-filters/reorder	POST	Updates sort order indices for saved filter presets in Redis.

9. Deployment & Build Configuration

- **Build Output:** Next.js compiles routes into optimized standalone JS chunks under .next.
- **Hosting Target:** Fully deployed on Vercel with automatic Upstash Redis Marketplace integration.

10. Known Limitations & Architecture Notes

- **Serverless Production Persistence:** Database records are fully persisted serverless across Vercel instances using @upstash/redis via Redis.fromEnv().
- **Zero-Config Local Fallback:** In local dev environments without Redis env vars, data safely falls back to the globalThis in-memory store.
- **Static Notifications:** The TopNavbar notification bell is a static UI action placeholder (no backend feed attached).